Exercise - 4 Solutions

4.1: Traditional Testing vs. ML Model Testing

Name at least four differences between testing traditional software and testing a machine learning model.

No.	Traditional software testing	ML model testing
1	Traditional software is **logic-driven**. Humans write rules and code logic.	ML systems are **data-driven**. The model learns behaviour from training examples.
2	Testing checks whether the written code behaves as expected.	Testing checks whether the learned behaviour generalizes to unseen data.
3	A unit test usually has a clear pass/fail result for one input.	A single ML input may fail, but the model is usually judged using aggregate metrics over many samples.
4	Code coverage is important, for example testing branches and functions.	Dataset coverage is important, for example whether the test set covers pedestrians, vehicles, lighting, weather, and ODD conditions.
5	Bugs usually come from wrong logic or implementation errors.	ML failures can come from bad data, class imbalance, label noise, overfitting, or distribution shift.
6	Once a bug is fixed, regression tests can check that the same bug does not return.	Even small changes in data, training, or model parameters can change model behaviour in unexpected ways.
7	Expected output is often exact and deterministic.	Output is often probabilistic, for example a confidence score or class probability.
8	Good test results usually mean the implemented logic works for tested cases.	Good test accuracy may still hide safety problems, for example low pedestrian recall despite high overall accuracy.

4.2: Test Oracles

In traditional software testing, a test oracle determines whether a test has passed or failed.

1. What plays the role of the test oracle in ML model testing?

→

Traditional software testing	ML model testing
A test oracle checks whether the program output is correct or incorrect.	The **ground-truth label** in the test dataset plays the role of the test oracle.

In ML testing, we compare the model prediction with the expected label from the dataset.

Example:

Image	Ground truth label	Model prediction	Test result
Image with pedestrian	Pedestrian = True	Pedestrian = True	Correct
Image with pedestrian	Pedestrian = True	Pedestrian = False	False negative
Image without pedestrian	Pedestrian = False	Pedestrian = True	False positive

So, for the CARLA perception models, the labels such as:

Task	Oracle
Pedestrian detector	`has_pedestrian` label
Traffic light detector	`has_traffic_light` label
Vehicle detector	`has_vehicle` label

act as the test oracle.

2. Why is defining a good test oracle particularly difficult for perception tasks?

→
In ML model testing, the test oracle is usually the ground-truth label in the test dataset. The model prediction is compared with this label to decide whether the prediction is correct or incorrect.

For perception tasks, defining a good oracle is difficult because the real world is visually complex. Objects may be small, far away, partially occluded, or difficult to distinguish. Labels can also be noisy or subjective because different annotators may make different decisions. In the CARLA system, a binary label such as has_pedestrian tells whether a pedestrian is present, but it does not fully describe whether the pedestrian is in the vehicle path or safety-critical. Therefore, the oracle may be incomplete for safety evaluation.

Distribution Shift and Data Quality

4.4: Distribution Shift Types

Consider the following three scenarios involving the CARLA autonomous driving model (trained on sunny daytime data):

For each scenario:
a) Identify which type of distribution shift applies (covariate shift, label shift, or concept shift). Justify your answer.
b) Explain what effect you would expect on model performance.
c) Suggest one mitigation strategy

1. The model is deployed during winter, when roads are often wet and the sun is at a low angle, creating glare in camera images.

→

Winter, wet roads, low sun, glare

Part	Answer
a) Shift type	**Covariate shift**

Justification	The input images change because of wet roads, reflections, glare, and winter lighting. However, the meaning of the labels stays the same: pedestrian is still pedestrian, vehicle is still vehicle, traffic light is still traffic light.
b) Expected effect	Model performance may drop because the images look different from sunny daytime training data. The model may miss pedestrians, vehicles, or traffic lights due to glare and reflections. False positives may also increase.
c) Mitigation	Add winter, wet-road, and glare images to the training and test sets. Also use data augmentation for brightness, contrast, glare, and reflections.

This is covariate shift because P(X) changes, but P(Y|X) stays mostly the same. The lecture describes covariate shift as a change in the input distribution, for example different weather or lighting.

2. A new city zone is added to the test route where 60 % of road users are cyclists - a class that was rare (< 5 %) in the original training data.

→

New city zone with 60% cyclists

Part	Answer
a) Shift type	**Label shift**
Justification	The class distribution changes. Cyclists were rare in the original training data, but now they appear very often in the new city zone.
b) Expected effect	The model may perform poorly on cyclists because it has seen very few cyclist examples during training. It may confuse cyclists with pedestrians or vehicles, or miss them completely.
c) Mitigation	Collect more cyclist data and retrain or fine-tune the model. The dataset should be rebalanced so cyclists are sufficiently represented.

This is a label shift because the proportion of classes changes. For example when more pedestrians appear in a school zone.

3. The city replaces all old traffic light housings with a new, slimmer design that the model has never seen.

→

New slimmer traffic light design

Part	Answer
a) Shift type	**Concept shift**
Justification	The relationship between the visual input and the label changes because the object still means “traffic light,” but it looks different from what the model learned before.
b) Expected effect	The traffic light detector may fail to recognize the new design. This can reduce recall, meaning real traffic lights may be missed. This is safety-critical because the vehicle may fail to stop at intersections.
c) Mitigation	Add images of the new traffic light design to the training data and fine-tune the traffic light detector. Also test the model specifically on the new design before deployment.

This can be treated as a concept shift because the model’s learned relationship between appearance and label no longer fully matches the new environment. The lecture gives new traffic sign designs as an example of concept shift.

4.5: ODD Coverage with k-Projection Coverage

You defined an ODD in Exercise Sheet 2. Now assess how well your test set covers this ODD using k-projection coverage.

1. Describe in your own words what k-projection coverage measures and why it is a useful metric for ODD coverage.

→

Term	Explanation
ODD	The set of conditions where the system is supposed to operate, for example weather, lighting, road type, traffic participants, etc.

Projection	Looking only at a subset of ODD dimensions instead of all dimensions at once.
k-projection coverage	Measures how many combinations of **k ODD dimensions** are covered by the test set.

For example, suppose the ODD dimensions are:

Dimension	Values
Weather	clear, rain, fog
Lighting	day, night
Scene type	urban, highway

Then:

k value	Meaning
k = 1	Checks whether each individual dimension value appears, for example clear, rain, fog, day, night.
k = 2	Checks whether pairs of dimensions appear, for example clear + day, rain + night, fog + urban.
k = 3	Checks whether triples of dimensions appear, for example clear + day + urban.

It is useful because safety problems often appear in **combinations** of conditions, not only single conditions. For example, “night” alone may be manageable and “rain” alone may be manageable, but “night + rain + pedestrian” may be much harder for the model.

ODD and dataset coverage should ensure that test sets cover the relevant ODD conditions, and k-projection coverage is suggested as a way to avoid testing every possible full combination.

2. Download the implementation from https://github.com/kkirchheim/odd-coverage and compute the k-projection coverage of your test set for k ∈ {1, 2, 3}.

→ (ref: ipynb for code)

3. How does coverage change with increasing k? What does this tell you about the adequacy of your test set?

→

k	What it checks	Result meaning
k = 1	Individual ODD values	Only **38.46%** of single ODD values are covered. This means many individual ODD conditions such as rain, fog, snow, night, glare, dirty camera, highways, etc. are missing.
k = 2	Pairwise combinations	Coverage drops to **14.95%**, meaning the test set covers only a small number of two-condition combinations, such as clear + day or urban + vehicle.
k = 3	Three-way combinations	Coverage drops further to **5.88%**, meaning very few three-condition combinations are represented.

→ This shows that the test set has limited ODD coverage. It mainly represents clear daytime urban CARLA scenes with a clean forward-facing camera. However, it does not sufficiently cover important ODD dimensions such as rain, fog, snow, night driving, glare, dirty camera conditions, highways, or construction zones.

The decreasing coverage means that the test set is not adequate for making broad safety claims across the full ODD defined in Exercise 2. It may be acceptable only for a narrow ODD: clear daytime urban driving. For more complex combinations, such as night + rain + pedestrian or fog + traffic light + dirty camera, the test set provides little or no evidence about model performance.

4.6: Designing a Test Suite from Safety Constraints

Refer back to Exercise 2.7 in Sheet 2, where you derived safety constraints from Unsafe Control Actions (UCAs). For each model-level constraint, design one test case using the following structure:

Constraint ID	Constraint	Test input description	Expected output	Pass criterion
SC-1	The planning module shall issue a brake command whenever the perception system detects a pedestrian within the stopping distance in the vehicle's path.	Test images containing pedestrians directly in front of the ego vehicle or crossing the road.	Pedestrian detector predicts **pedestrian present**.	Pedestrian recall should be high. The model should correctly detect most pedestrians in safety-critical positions.
SC-2	The system shall detect and correctly classify traffic lights and command a stop when the signal is red.	Test images from intersections where traffic lights are visible.	Traffic light detector predicts **traffic light present**.	Traffic light recall should be high. The model should not miss visible traffic lights at intersections.
SC-3	The system shall detect crossing vehicles and issue braking or yielding commands when a collision risk exists.	Test images containing vehicles ahead, crossing, or near intersections.	Vehicle detector predicts **vehicle present**.	Vehicle recall should be high. The model should detect vehicles that could affect the ego vehicle's path.
SC-6	The system shall require confidence thresholds and temporal consistency before issuing emergency braking commands.	Test images with no pedestrians, vehicles, or traffic lights in the driving path.	The corresponding detector predicts **object absent**.	False positive rate should be low. The model should not frequently detect objects when none are present.

4.7: Per-Class Evaluation Evaluate each of your three models on the provided test set.

1. Compute and report precision, recall, and F1-score for each model.

→

Model	Recall
Pedestrian Detector	0.3385
Traffic Light Detector	0.9717
Vehicle Detector	0.8893

2. Plot a confusion matrix for each model.

→

(ref: ipynb for code and confusion matrix)

3. Which model has the lowest recall? Is this the model you expected to be hardest based on your hazard analysis?

→

The **Pedestrian Detector** has the lowest recall: **0.3385**.

Yes, this is the model I expected to be hardest. Pedestrian detection is difficult because pedestrians are often:

Reason	Explanation
Small in the image	Pedestrians may appear far away and occupy only a few pixels.

Visually variable	Pedestrians have different poses, clothes, sizes, and orientations.
Sometimes occluded	They may be partly hidden behind vehicles or other objects.
Less frequent	The dataset likely contains fewer pedestrian examples compared to vehicles or traffic lights.

4. Based on a safety argument: what minimum recall would you require for the pedestrian model before considering deployment? Justify your threshold.

→

From a safety perspective, I would require the pedestrian detector to achieve at least **99% recall** before considering deployment, and only within the validated ODD. The reason is that false negatives are the most dangerous error for pedestrian detection. If the model misses a real pedestrian, the planning module may not brake, which can lead directly to hazard **H-1** and potentially to injury or death. Therefore, the recall threshold must come from the hazard analysis, not from the current model performance. Since the current pedestrian recall is only about **33.85%**, the model is clearly not suitable for deployment.